CS69201 · COMPUTING LAB · TERM PROJECT

Terminator

Build your own terminal, one system call at a time.

```
user@terminator> _
```

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

What happens when you press Enter?

```
user@terminator> ls *.txt | wc -l
3
```

After this project, you will have built every step of the answer yourself.

THE BIG PICTURE

Three layers between you and the hardware

Terminal	Draws the window, reads keys, shows text.	You build · X11
Shell	Understands what you typed, and runs it.	You build · C
Kernel	Creates processes, moves bytes, delivers signals.	Linux · syscalls

bash is a shell. GNOME Terminal is a terminal. **Terminator is both.**

WHAT YOU WILL BUILD

Terminator, in one slide

01
X11 GUI & tabs
Your own window

02
Commands
fork, execvp, wait

03
Multiline input
Quotes, \ and $(...)

04
Redirection
< and > via dup2

05
Pipes
cmd | cmd | cmd

06
multiWatch
Parallel live output

07
Hotkeys
User-configurable

08
Signals & jobs
Ctrl+C, Ctrl+Z, fg, bg

09
History
Shared, Ctrl+R search

10
Auto-complete
Tab finishes names

11
Bash scripts
Your own interpreter

UNDER ALL
Two modes
Threads or processes

Six acts, one command's journey

ACT 1	**The window**	How do keystrokes get in and text get out?
ACT 2	**Running a command**	How does a line of text become a running program?
ACT 3	**Plumbing**	How do programs read files and talk to each other?
ACT 4	**Many things at once**	How do tabs and multiWatch run side by side?
ACT 5	**Taking control**	How do Ctrl+C and Ctrl+Z work without a real TTY?
ACT 6	**Feels like bash**	Quotes, hotkeys, history, completion and scripts.

ACT 1 / 6

The window

How keystrokes get in, and text gets out.

Covers Task 1 · X11 GUI and tabs

ACT 1 · THE WINDOW

X11 is a conversation with a server

Terminator
X client: your code

→ draw requests
← input events

X server
Owns the display

→ pixels
← keys, clicks

Screen & keyboard
The hardware

Four calls get you started. Everything else arrives as an **event**.

`XOpenDisplay`
Connect to the server

`XCreateSimpleWindow`
Make a window

`XNextEvent`
Get the next event

`XDrawString`
Draw a line of text

ACT 1 · THE WINDOW

The window lives inside an event loop

```
while (running) {
    poll(X fd, tab pipes); // sleep
    if (key)    edit input line;
    if (output) append to buffer;
    redraw what changed;
}
```

Sleep, don't spin

Wait on the X connection and your pipes together with select() or poll().

Never freeze the window

A long-running command must not block keys or redraws.

One thread talks to X

Keep Xlib calls in one thread, or call XInitThreads() first.

ACT 1 · THE WINDOW

The screen is a drawing of your buffers

Edit, then redraw

Keys and output change the buffer in memory; the window is redrawn from it.

One buffer per tab

The tab bar only decides which buffer gets drawn.

Hidden tabs keep working

Their output keeps filling their buffer while you look elsewhere.



ACT 2 / 6

Running a command

How a line of text becomes a running program.

Covers Task 2 · Built-in and external commands

ACT 2 · RUNNING A COMMAND

Every command: fork, exec, wait

STEP 1

fork()

Clone the shell. Now there are two identical processes.

→

STEP 2 · CHILD

execvp()

The child replaces itself with the program, such as ls.

→

STEP 3 · PARENT

waitpid()

The shell waits for the child, then shows the prompt.

```
pid_t pid = fork();
if (pid == 0) { // child
    execvp(argv[0], argv);
    _exit(127); // only reached if exec failed
}
waitpid(pid, &status, 0); // parent
```

ACT 2 · RUNNING A COMMAND

Some commands run inside the shell

External

`ls · gcc · sort · ./myprog`

Programs stored as files. Run in a child with fork() + execvp().

Built-in

`cd · exit · history · jobs · fg · bg · source · multiWatch`

Run inside the shell, because they change the shell itself.

Why is cd a built-in? A child that calls chdir() changes only its own directory, and then it exits.

> **$?** keeps the last exit status. Scripts depend on it.

> **Unknown command?** Print an error; the tab keeps working.

ACT 3 / 6

Plumbing

How programs read files and talk to each other.

Covers Task 4 · I/O redirection, and Task 5 · Pipes

ACT 3 · PLUMBING

Every process has an fd table

fd	points to
0 stdin	keyboard (or a pipe)
1 stdout	the screen (or a pipe)
2 stderr	the screen
3	infile.txt, after open()

Programs only know numbers.

Point fd 0 or 1 somewhere else before exec, and the program reads or writes there without ever knowing.

`dup2(fd, 0) → fd is the new stdin`

ACT 3 · PLUMBING

Redirection is rewiring before exec

```
user@terminator> sort < in.txt > out.txt
```

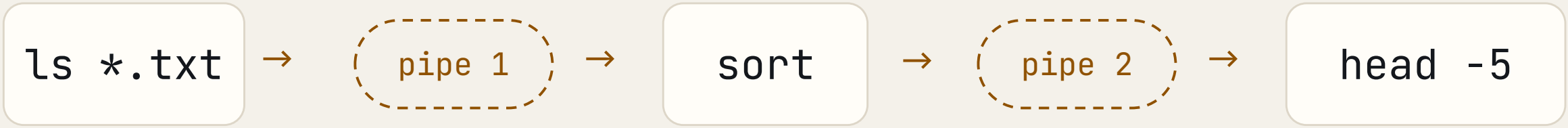
1. fork() a child.
2. In the child, open in.txt and dup2() it onto fd 0.
3. Open out.txt and dup2() it onto fd 1.
4. execvp("sort"). sort never knows.

fd	before	after
0	keyboard	in.txt
1	screen	out.txt
2	screen	screen

> Rewire in the child only. In the shell it would break the shell, and in thread mode, every tab.

ACT 3 · PLUMBING

A pipe joins one program's output to the next one's input



N−1 pipes, N children

Create the pipes, fork every child, then wait for all of them.

Close unused ends

One forgotten write end, and the reader never sees EOF.

One process group

So Ctrl+C later reaches the whole pipeline at once.

ACT 4 / 6

Many things at once

Tabs, shared state, and watching many pipes.

Covers execution modes, Task 1 (tabs) and Task 6 · multiWatch

ACT 4 · MANY THINGS AT ONCE

Two ways to run a tab

`./terminator`

Thread mode (default)

- One process; each tab is a thread
- Tabs share memory: lock shared state
- Cheap to create and switch
- chdir, dup2 and signals hit every tab

`./terminator useProcess=True`

Process mode

- GUI is the parent; each tab is forked
- Tabs talk to the GUI through pipes
- Shared state lives in shared memory
- A crashing tab can't take others down

Both modes must **behave the same**. The user should not be able to tell them apart.

ACT 4 · MANY THINGS AT ONCE

What tabs share, and what they keep

Shared by every tab

- Command history
- Clipboard

Type in tab 1, press ↑ in tab 2, and it is there.

Private to each tab

- Current directory
- Input line and output buffer
- Running and stopped jobs

`Mutex`
Lets one thread in at a time

`Semaphore`
A counter; works across processes

`Shared memory`
Same bytes in many processes

Your call: how you lay out and lock shared state. Explain it in your design document.

ACT 4 · MANY THINGS AT ONCE

One sleeper, many file descriptors

- `X connection`
- `tab 1 output`
- `tab 2 output`
- `multiWatch: cmd1`
- `multiWatch: cmd2`

→

`select() / poll()`

Sleeps until any of them has data, then says which.

→

Handle what's ready

Read it, update a buffer, go back to sleep.

Busy loop: read, read, read…

100% CPU, and the fan agrees.

poll(): sleep, wake, handle, sleep.

Near 0% CPU when idle.

ACT 4 · MANY THINGS AT ONCE

multiWatch: many commands, one view

1. One pipe() per command, then fork() each one.
2. Child: dup2() the pipe onto stdout, then exec.
3. Parent: poll() all read ends; print each chunk as it arrives.
4. Ctrl+C: kill() every child. The shell keeps running.

Unlike watch "cmd1 && cmd2", which runs them one after another, these run in parallel.

The output format is graded exactly.

```
user@terminator> multiWatch ["date +%T", "ls | wc -l"]
"date +%T", 1790860020:
-------------------------------------------------------
18:37:00
-------------------------------------------------------
"ls | wc -l", 1790860020:
-------------------------------------------------------
3
-------------------------------------------------------
^C
```

ACT 5 / 6

Taking control

Stopping, pausing and resuming running programs.

Covers Task 8 · Signals and job control

ACT 5 · TAKING CONTROL

Ctrl+C, without a real terminal

You press
Ctrl+C in tab 1

→

X11 event
KeyPress with ControlMask

→

Shell
Finds tab 1's foreground job

→

`kill()`
SIGINT to the job's process group

↓

Prompt
Returns.
Terminator lives on.

No TTY here
In a normal terminal, the kernel sends SIGINT for you. Here the GUI only sees a key, so your shell sends the signal.

Only the job is hit
Terminator itself, and other tabs' jobs, must never receive the signal.

ACT 5 · TAKING CONTROL

Job control: one table of moves

Input	From	To	Signal
Ctrl+C	Foreground	Ended	SIGINT
Ctrl+Z	Foreground	Stopped	SIGTSTP
bg %n	Stopped	Background	SIGCONT
fg %n	Stopped or bg	Foreground	SIGCONT
cmd &	(new)	Background	none

> A job is a process group, so one signal reaches every process in a pipeline. Use **jobs** to list them.

```
> sleep 100
^Z
[1]+ Stopped sleep 100
> bg %1
[1]+ sleep 100 &
> fg %1
sleep 100
^C
```

ACT 6 / 6

Feels like bash

Parsing, hotkeys, history, completion and scripts.

Covers Tasks 3, 7, 9, 10 and 11

ACT 6 · FEELS LIKE BASH

From raw text to a runnable command

Read
Until the input is complete

→

Split
Into words, honouring quotes

→

Expand
$VAR, $?, $(...), innermost first

→

Build
Pipelines, redirections, if-blocks

→

Run
The engine from Acts 2 to 4

`'single'`
Everything literal, even $HOME.

`"double"`
Keeps spaces; still expands $VAR and $(...).

`\ or open quote`
Show the > prompt and keep reading.

```
user@terminator> echo "Files in $(basename $(pwd)): $(ls | \
> wc -l)"
Files in mydir: 3
```

ACT 6 · FEELS LIKE BASH

Hotkeys: bash defaults, user's choice

Action	Default key
Move in the line	← / →
Start / end of line	Ctrl+A / Ctrl+E
Browse history	↑ / ↓
Delete to start / end	Ctrl+U / Ctrl+K
Delete previous word	Ctrl+W
Clear the screen	Ctrl+L
Close tab / send EOF	Ctrl+D
Copy / paste	Ctrl+Shift+C / V

```
# ~/.terminatorrc (one option)
bind clear-screen Ctrl+Shift+L
bind delete-word Alt+Backspace
```

- Change keys without recompiling
- Custom bindings survive a restart
- Bad binding: warn, keep the default
- Config file, command or prompt: you decide

ACT 6 · FEELS LIKE BASH

History you can search

10,000 commands, on disk

Like ~/.bash_history, it survives a restart.

history shows 10,00

The most recent ones.

Shared by every tab

The shared state from Act 4, in action.

`Ctrl+R → Enter search term:`

1. **Exact match?** Print the most recent one.
2. **Else:** the command(s) with the longest common substring, if it is longer than 2 characters.
3. **Else:** "No match for search term in history"

ACT 6 · FEELS LIKE BASH

Tab finishes file names

One match
Complete it. The rest of the line stays as typed.

Several matches
Extend to the longest common prefix, then show a numbered list.

No match
Do nothing.

```
# this directory holds abc.txt, abcd.txt and def.txt
user@terminator> ./myprog de [Tab]
user@terminator> ./myprog def.txt abc [Tab]
1. abc.txt   2. abcd.txt [press 1]
user@terminator> ./myprog def.txt abc.txt
```

It completes file names, not commands, so it works even if ./myprog does not exist.

ACT 6 · FEELS LIKE BASH

Scripts reuse everything you built

A script is the lines you would have typed, read from a file.

Must work

- ./run.sh, bash run.sh, source run.sh
- Variables, $1…$9, $#, $?
- Nested if / elif / else with [ ] tests
- Pipes, redirection, $(…), exit n

Not required: loops and functions.

Interpret it yourself. Handing the file to /bin/bash does not count.

```
user@terminator> cat check.sh
#!/bin/bash
if [ -f "$1" ]; then
    echo "File exists, compiling..."
    gcc "$1" -o myprog
else
    echo "Source file missing!"
fi
user@terminator> ./check.sh myprog.c
File exists, compiling...
```

WRAP-UP

A suggested path to completing the assignment

WEEK 1

One window, one shell

X11 loop, fork and exec, built-ins, $?

WEEK 2

Plumbing and parsing

Redirection, pipes, quotes, multiline input

WEEK 3

Many at once

Tabs in both modes, shared history, multiWatch, signals, jobs

WEEK 4

Feels like bash

Hotkeys, Ctrl+R, auto-complete, scripts

FINAL

Write it up

README, design document, demo commands

Tip: build a shell engine that doesn't care where text comes from. Then the GUI, tabs and scripts plug into it.

WRAP-UP

What to submit

Submit your **GitHub repository link** through the Moodle form. The repository must contain:

Source code
The complete Terminator implementation.

Makefile
make builds ./terminator with the X11 flags, without errors.

README
What it is, dependencies, how to build and run both modes, features, known limitations.

Design document
GUI and event loop, both modes and synchronisation, jobs, IPC, data structures, signals.

Anything that helps
Test scripts, sample config, demo commands, screenshots.

Deadline

3 November 2026, 11:59 PM

THAT'S THE MAP

Questions?

THAT'S THE MAP

Questions?

Start this week: get ls running in your own window.

```
user@terminator> ls
abc.txt  def.txt  myprog.c
```